pypomp: Inference for partially observed Markov process models in Python with JAX
DRAFT IN PROGRESS

Aaron J. Abkemeier*

Jun Chen*

Kevin Tan

Jesse Wheeler

Bo Yang

Kunyang He

Jonathan Terhorst

Aaron A. King

Edward L. Ionides

***These authors contributed equally**

Editor:

Abstract

Methods for fitting and evaluating partially observed Markov process (POMP) mechanistic models are useful in a variety of fields such as epidemiology, ecology, and finance, but are often hindered by high computational demands. We introduce **pypomp**, a high-performance Python package for statistical inference using POMP models. Designed for complex stochastic dynamic systems, **pypomp** implements plug-and-play, likelihood-based algorithms including sequential Monte Carlo and iterated filtering. By leveraging GPU acceleration, **pypomp** significantly outperforms its predecessor, the R package **pomp**. Models that previously required months to fit now complete in days, enabling the use of formerly impractical models. Furthermore, **pypomp** uses JAX to perform gradient descent on differentiable particle filters, achieving likelihood maximization beyond the practical limits of iterated filtering alone. The package provides a comprehensive interface that streamlines model construction, fitting, and analysis, allowing practitioners to develop and evaluate complex models with minimal implementation overhead.

Keywords: pomp, JAX, GPU acceleration, likelihood-based inference

1 Introduction

Partially Observable Markov Process (POMP) models, also known as state-space models or hidden Markov models, provide a flexible mechanistic framework for modeling time-series dynamic systems where the underlying state process is not directly observed. Methods such as particle filtering (Arulampalam et al., 2002) and iterated filtering (Ionides et al.,

2006, 2015) have been developed to perform maximum likelihood estimation and evaluation for POMP models. A defining strength of these methods is that they are performed by simulating from the process model instead of evaluating exact transition probabilities (a property known as plug-and-play (Bretó et al., 2009)), which makes them suitable for examining a wide range of models unconstrained by a need for mathematical convenience. Consequently, POMP models find extensive application in epidemiology (Mietchen et al., 2024; Fox et al., 2022; Wen et al., 2024), ecology (Auger-Méthé et al., 2021; Marino et al., 2019; Blackwood et al., 2013), finance (Bretó, 2014), and other domains where mechanistic modeling is desired.

Thus far, the `pomp` package ecosystem in R has provided a basis for POMP modeling of several types. The `pomp` package is designed for standard POMP models, while its extension packages `panelPomp`, `spatpomp`, and `phyloPomp` further enhance its capabilities for panel, spatio-temporal, and phylodynamic data analysis, respectively (King et al., 2016; Bretó et al., 2025; Asfaw et al., 2024).

While powerful, statistical inference for POMP models poses substantial computational challenges, which existing packages struggle with due to their lack of support for GPU acceleration and automatic differentiation. To address this, we introduce the package `pypomp` (Abkemeier et al., 2024), a JAX-based (Bradbury et al., 2018) Python implementation of core `pomp` and `panelPomp` algorithms. [AA: Need to change "90 times faster" to something more specific after we more rigorously compare pypomp with pomp. In addition, I think we should include the @korevaar2020 data set fit in this paper, as it provides perhaps the strongest motivation for developing pypomp.] By leveraging GPU acceleration, the package allows users to run methods up to 90 times faster than the CPU-based alternatives, enabling plug-and-play likelihood-based inference on data sets that would otherwise be intractably large. This is demonstrated in Section [AA: add this section], where we fit a panel POMP model to the Korevaar et al. (2020) England and Wales measles dataset, which contains over one million observations. While this would normally take over 5 months in `pomp`, `pypomp` can finish the job in 41 hours [AA: revise these numbers as appropriate]. In addition to GPU acceleration, `pypomp` uses JAX to perform gradient descent on differentiable particle filters (Tan et al., 2024), which can further accelerate parameter estimation.

The remainder of this paper is organized as follows. Section 3 introduces the embedded methodologies. **?@sec-analysis** presents data analysis workflows and benchmarking results. Section 4 concludes with a discussion of future work.

2 POMP Modeling

A POMP model is defined by an initial density $f_{X_0}(x_0;\theta)$, a transition density $f_{X_n|X_{n-1}}(x_n \mid x_{n-1};\theta)$, and a measurement density $f_{Y_n|X_n}(y_n \mid x_n;\theta)$.

3 POMP Inference Methods in `pypomp`

`pypomp` implements several core likelihood-based inference methods, all of which leverage the plug-and-play property (Ionides et al., 2006) and JAX-powered acceleration. These methods are implemented as methods of the `Pomp` class and modify the object in place by appending results to `results_history`.

1. **Particle Filter (`pfilter`)**: A standard sequential Monte Carlo (SMC) algorithm for likelihood evaluation and state estimation.
2. **Iterated Filtering (`mif`)**: An implementation of the IF2 algorithm (Ionides et al., 2015) for maximum likelihood estimation. It combines particle filtering with sequential random walk perturbations and a cooling schedule to explore the parameter space.
3. **Automatic Differentiation (`mop` and `train`)**: `pypomp` includes a differentiable particle filter (MOP-α, Tan et al. (2024)) that enables the use of JAX's automatic differentiation. The `train()` method uses these gradients for local optimization.

The following example demonstrates a typical inference workflow: starting with global exploration via IF2, followed by gradient-based local refinement, and concluding with a final likelihood evaluation. While users can define custom models by providing JAX-compatible Python functions for each component, `pypomp` we demonstrate the API with the premade S&P500 model.

```
import pypomp as pp
import jax

spx_obj = pp.models.spx()

key = jax.random.key(1)
key, subkey = jax.random.split(key)

rw_sd = pp.RWSigma(
    sigmas={
        "mu": 0.02,
        "kappa": 0.02,
        "theta": 0.02,
        "xi": 0.02,
        "rho": 0.02,
        "V_0": 0.1,
    },
    init_names=["V_0"],
)
spx_obj.mif(rw_sd=rw_sd, J=1000, M=10, a=0.5, key=subkey)

eta = {name: 0.001 for name in spx_obj.canonical_param_names}
spx_obj.train(J=1000, M=10, eta=eta, optimizer="Adam")

spx_obj.pfilter(J=1000, reps=10, key=subkey)

print(spx_obj.traces().head())
```

```
W0515 16:14:05.953362  956213 cpp_gen_intrinsics.cc:74] Empty bitcode string provided for e

   theta_idx  iteration method        logLik        mu     kappa     theta  \
```

```
0          0          0    mif           NaN  0.000368  0.031400  0.000112
1          0          1    mif  11841.455078  0.000309  0.063548  0.000080
2          0          2    mif  11851.680664  0.000138  0.041170  0.000104
3          0          3    mif  11846.740234  0.000310  0.035668  0.000105
4          0          4    mif  11836.334961  0.000030  0.044204  0.000095
```

```
         xi       rho       V_0
0  0.002270 -0.738000  0.000059
1  0.001374 -0.865430  0.000066
2  0.001519 -0.768029  0.000059
3  0.001537 -0.769967  0.000065
4  0.001162 -0.939040  0.000064
```

Each method call generates a result object which contains execution time, parameter traces, and log-likelihood estimates. These objects provide helper functions to view the results, such as the parameter and log-likelihood traces across multiple method calls.

4 Discussion

References

Aaron Abkemeier, Jun Chen, Edward Ionides, Jesse Wheeler, and Kevin Tan. Pypomp, 2024.

M.S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, February 2002. ISSN 1053587X. doi: 10.1109/78.978374.

Kidus Asfaw, Joonha Park, Aaron A. King, and Edward L. Ionides. spatPomp: An R package for spatiotemporal partially observed Markov process models. *Journal of Open Source Software*, 9(104):7008, December 2024. ISSN 2475-9066. doi: 10.21105/joss.07008.

Marie Auger-Méthé, Ken Newman, Diana Cole, Fanny Empacher, Rowenna Gryba, Aaron A. King, Vianey Leos-Barajas, Joanna Mills Flemming, Anders Nielsen, Giovanni Petris, and Len Thomas. A guide to state-space modeling of ecological time series. *Ecological Monographs*, 91(4):e01470, 2021. doi: 10.1002/ecm.1470. URL `https://doi.org/10.1002/ecm.1470`.

J. C. Blackwood, D. G. Streicker, S. Altizer, and P. Rohani. Resolving the roles of immunity, pathogenesis, and immigration for rabies persistence in vampire bats. *Proceedings of the National Academy of Sciences of the United States of America*, 110(51):20837–20842, December 2013. doi: 10.1073/pnas.1308817110. URL `https://doi.org/10.1073/pnas.1308817110`.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, ake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018.

Carles Bretó. On idiosyncratic stochasticity of financial leverage effects. *Statistics & Probability Letters*, 91:20–26, 2014. doi: 10.1016/j.spl.2014.04.003. URL https://doi.org/10.1016/j.spl.2014.04.003.

Carles Bretó, Daihai He, Edward L. Ionides, and Aaron A. King. Time series analysis via mechanistic models. *The Annals of Applied Statistics*, 3(1):319–348, March 2009. ISSN 1932-6157, 1941-7330. doi: 10.1214/08-AOAS201.

Carles Bretó, Jesse Wheeler, Aaron A. King, and Edward L. Ionides. panelPomp: Analysis of Panel Data via Partially Observed Markov Processes in R, May 2025.

S. J. Fox, M. Lachmann, M. Tec, R. Pasco, S. Woody, Z. Du, X. Wang, T. A. Ingle, E. Javan, M. Dahan, K. Gaither, M. E. Escott, S. I. Adler, S. C. Johnston, J. G. Scott, and L. A. Meyers. Real-time pandemic surveillance using hospital admissions and mobility data. *Proceedings of the National Academy of Sciences*, 119(7):e2111870119, 2022. doi: 10.1073/pnas.2111870119. URL https://doi.org/10.1073/pnas.2111870119.

E. L. Ionides, C. Breto, and A. A. King. Inference for nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 103(49):18438–18443, December 2006. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.0603181103.

Edward L. Ionides, Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *Proceedings of the National Academy of Sciences*, 112(3):719–724, January 2015. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.1410597112.

Aaron A. King, Dao Nguyen, and Edward L. Ionides. Statistical Inference for Partially Observed Markov Processes via the *R* Package **pomp**. *Journal of Statistical Software*, 69(12), 2016. ISSN 1548-7660. doi: 10.18637/jss.v069.i12.

Hannah Korevaar, C. Jessica Metcalf, and Bryan T. Grenfell. Structure, space and size: Competing drivers of variation in urban and rural measles transmission. *Journal of The Royal Society Interface*, 17(168):20200010, July 2020. doi: 10.1098/rsif.2020.0010.

J. A. Jr. Marino, S. D. Peacor, D. B. Bunnell, H. A. Vanderploeg, S. A. Pothoven, A. K. Elgin, J. R. Bence, J. Jiao, and E. L. Ionides. Evaluating consumptive and nonconsumptive predator effects on prey density using field time-series data. *Ecology*, 100(3):e02583, March 2019. doi: 10.1002/ecy.2583. URL https://doi.org/10.1002/ecy.2583.

Matthew S. Mietchen, Erin Clancey, Corrin McMichael, and Eric T. Lofgren. Estimating sars-cov-2 transmission parameters between coinciding outbreaks in a university population and the surrounding community, Jan 2024. URL https://doi.org/10.1101/2024.01.10.24301116.

Kevin Tan, Giles Hooker, and Edward L. Ionides. Accelerated Inference for Partially Observed Markov Processes using Automatic Differentiation, July 2024.

L. Wen, Y. Yin, Q. Li, Z. Peng, and D. He. Modeling the co-circulation of influenza and covid-19 in hong kong, china. *Advances in Continuous and Discrete Models*, 2024

(1):1–9, 2024. doi: 10.1186/s13662-024-03830-7. URL https://doi.org/10.1186/s13662-024-03830-7. Article 32.

5 Appendix

[AA: Include imporant numerical results here? Or refer to the quant repo?]